openMSX Setup Guide

Contents

1. Introduction
    1.1 New Versions of this Document
    1.2 Purpose
    1.3 Revision History
2. Machines and Extensions
3. System ROMs
    3.1 C-BIOS
    3.2 Dumping ROMs
        3.2.1 Tools
        3.2.2 Legal Issues
    3.3 Downloading ROMs
    3.4 Installing ROMs
        3.4.1 ROM Locations
        3.4.2 How openMSX knows which ROM files to use
        3.4.3 How to handle split ROMs
4. Palcom Laserdiscs
5. User Preferences
6. Performance Tuning
    6.1 OpenGL
    6.2 Various Tuning Tips
7. Writing Hardware Descriptions
8. Contact Info

1. Introduction

1.1 New Versions of this Document

The latest version of the openMSX manual can be found on the openMSX home page:

http://openmsx.org/manual/

You can also use this URL to get up-to-date versions of the hyper links if you printed out this manual.

1.2 Purpose

This guide is about openMSX, the open source MSX emulator that tries to achieve near-perfect emulation by using a novel emulation model. You can find more information about openMSX on the openMSX home page. You can also download the emulator itself from there.

This guide describes the setup of openMSX. After installation, openMSX is ready to run using C-BIOS and the default settings. In this guide you can read how to configure openMSX to emulate actual MSX machines (such as Panasonic FS-A1GT). It also describes how you can have openMSX start up with your personal settings, how you can configure openMSX and your system for optimal performance, and several other configuration related topics.

Disclaimer: We do not claim this guide is complete or even correct. What you do with the information in it is entirely at your own risk. We just hope it helps you enjoy openMSX more.

1.3 Revision History

For the revision history, please refer to the commit log.

2. Machines and Extensions

We use the word *machine* to refer to a specific MSX model. For example, the Sony HB-75P is a machine. openMSX does not have a fixed machine hardcoded into it. Instead, many different MSX machines can be emulated. The details of a machine are described in an XML file. This file describes how much memory a machine has, what video processor it has, in which slots its system ROMs are located, whether the machine has a built-in disk drive etc. openMSX reads the machine description XML and will then emulate exactly that MSX machine, which can be anything from an MSX1 with 16 kB of RAM to the Panasonic FS-A1GT MSX turboR.

The openMSX distribution contains XML files describing many existing MSX models. You can find them in the `share/machines` directory. If you want to run one of those machines, you also need the system ROMs for that machine. See the next chapter for more information on system ROMs. You can also create your own machine descriptions, to expand existing MSX models or to create your own fantasy MSX. There are currently some of such fantasy MSX machines, based on real MSX machines, shipped with openMSX. Examples of such machines are "Boosted_MSX2_EN" (a European MSX2 with loads of hardware built-in) and "Boosted_MSX2+_JP" (a Japanese MSX2+ with loads of hardware built-in). You can find some more information about them in their accompanying txt file in `share/machines/`. More about creating fantasy MSX machines in a later chapter.

An *extension* is a piece of MSX hardware that can be inserted into a cartridge slot to extend the capabilities of an MSX. Examples of extensions are the Panasonic FMPAC, the Sunrise IDE interface and an external 4MB memory mapper. Extensions, like machines, are described in XML files. You can find a lot of predefined extensions in the `share/extensions` directory. Some extensions need ROM images in order to run, similar to system ROMs.

In general, the XML files that describe the hardware configuration are called "hardware configuration XML files".

If you want to be able to run a combination of a machine and plugged in extensions at a later time, you can store this combination as a setup with the GUI via *Main menu bar → Machine → Save setup* or the `store_setup` command.

3. System ROMs

An MSX machine consists of a lot of hardware, but also contains some software. Such software includes the BIOS, MSX-BASIC, software controlling disk drives and built-in applications (firmware). openMSX emulates the MSX hardware, but it needs MSX system software to emulate a full MSX system. Because the internal software is located in ROM chips, it is referred to as *system ROMs*.

The software in the system ROMs, like most software, is copyrighted. Depending on your local laws, there are certain things you are allowed to do with copyrighted software and certain things you are not allowed to do. In this manual, a couple of options are listed for providing system ROMs to your openMSX installation. It is up to you, the user, to select an option that is legal in your country.

3.1 C-BIOS

C-BIOS stands for "Compatible BIOS". It aims to be compatible with the MSX BIOS found in real MSX machines, but it was written from scratch, so all copyrights belong to its authors. BouKiChi, the original author of C-BIOS, was kind enough to allow distribution of C-BIOS with openMSX. When Reikan took over maintenance of C-BIOS, the license was changed to give users and developers even more freedom in using C-BIOS. Later still, C-BIOS was moved to a SourceForge.net project, with several new maintainers. Every now and then, an updated version of C-BIOS is released. You can wait for it to be included in the next openMSX release, or download it directly from the C-BIOS web site.

C-BIOS can be used to run most MSX1, MSX2 and MSX2+ cartridge-based games. It does not include support for MSX-BASIC or disk drives yet, so software that comes on tape, disk or any other media than ROM cartridges will not run on standard openMSX C-BIOS machines.

openMSX contains several machine configurations using C-BIOS. The machine `C-BIOS_MSX1` is an MSX1 with 64 kB RAM. The machine `C-BIOS_MSX2` is an MSX2 with 512 kB RAM and 128 kB VRAM. The machine `C-BIOS_MSX2+` is an MSX2+ with 512 kB RAM, 128 kB VRAM and MSX-MUSIC. The latter is the default machine for openMSX after installation, so if you change nothing to the openMSX configuration, then `C-BIOS_MSX2+` is the machine that will be booted. The mentioned machines have a US English (international) keyboard layout and character set and run at 60Hz (like NTSC) interrupt frequency. Since C-BIOS 0.25, some localized versions are also available: Japanese, European (like US, but 50Hz), and Brazilian. You can easily recognize them.

It is always legal for you to run the C-BIOS ROMs in openMSX. You are allowed to use C-BIOS and its source code in various other ways as well, read the C-BIOS license for details. It is located in the file `README.cbios` in the Contrib directory.

3.2 Dumping ROMs

If you own a real MSX machine, you can dump the contents of its system ROMs to create ROM images you can run in openMSX. This way, you can emulate the MSX machines you are familiar with.

3.2.1 Tools

The easiest way to dump system ROMs is to run a special dumping tool on your real MSX, which copies the contents of the system ROMs to disk. Sean Young has made such tools, you can find the tools and documentation on BiFi's web site. These tools can also be used to dump cartridge ROMs, which may be useful later, if you want to use certain extensions or play games.

3.2.2 Legal Issues

Using ROMs dumped from machines you own is generally speaking not frowned upon in the MSX community. When the MSX machine was bought in a shop years ago, you or the person that originally bought it paid money for the MSX machine. A small part of that money paid for the software in the system ROMs. However, we are no legal experts, so it is up to you to check whether it is legal in your country to use dumped ROMs of machines you own.

3.3 Downloading ROMs

Some WWW and FTP sites offer MSX system ROMs as a download. Some MSX emulators include system ROMs in their distribution. Downloaded system ROMs can be used in the same way as system ROMs you dumped yourself, see the previous section.

It may be illegal in your country to download system ROMs. Please inform yourself of the legal aspects before considering this option. Whatever you decide is your own responsibility.

3.4 Installing ROMs

If you want to emulate real MSX machines next to the default C-BIOS based machines, you will have to install system ROMs that did not come with openMSX. This section explains how to install these, once you obtained them in one of the ways that are explained in the previous sections.

3.4.1 ROM Locations

The easiest way is to copy the ROM files in a so-called file pool: a special directory where openMSX will look for files (system ROMs, other ROMs, disks, tapes, etc.). The default file pool for system ROMs is

the `systemroms` sub directory. The best way is to make a `systemroms` sub directory in your own user directory, which is platform dependent:

Platform	User directory
Windows (e.g. Windows 7)	`C:\Users\<user name>\Documents\openMSX\share` whereby `<user name>` stands for your Windows login name
Unix and Linux	`~/.openMSX/share`

Please note that the path part which comes before `share` can be overridden by setting the `OPENMSX_HOME` environment variable, see the chapter about User Preferences.

That way, you do not need special privileges. Furthermore, the (Windows) installer won't touch them for sure.

A template for the `systemroms` sub directory is present in the installation directory of openMSX, which is also platform dependent:

Platform	Typical openMSX file pool installation directory
Windows (any version)	`C:\Program Files\openMSX\share`
Unix and Linux	`/opt/openMSX/share` or `/usr/share/openmsx`

The quickest way to see where openMSX is searching for system ROMs on your installation is via the GUI under *Main menu bar → Machine → Test MSX hardware*. At the bottom of this dialog you can find buttons to quickly open a (native) file browser on the locations where the ROMs are searched for, both for the user folder and the system wide folder. The main function of this window is to verify whether your system ROMs have been installed properly.

In short: you can just copy all your system ROMs to the `share/systemroms` directory of your user account. The ROM files can be zipped (or gzipped), but only one file can be in a ZIP file. If multiple ROM files are in a single ZIP file, openMSX will not find them. The directory structure below `share/systemroms` is not relevant, openMSX will search it completely.

More info about file pools is in the documentation of the `filepool` command. If you can't get this working, please read one of the next sections.

For advanced users, it is also possible to let openMSX load a specific set of ROM images for a machine, independent of any file pool or the checksums of the ROM images. For that you copy the ROM file with the name and path as mentioned in the hardware configuration XML file that describes the machine, relative to the path of that machine description file. For example, if you dumped the ROMs of a Philips NMS 8250 machine, copy them to `share/machines`, because in the machine description file (in `share/machines/Philips_NMS_8250.xml`) the name of the ROMs is like this: `nms8250_msx2sub.rom`. We recommend to not use this feature, but use the file pools as mentioned above.

3.4.2 How openMSX knows which ROM files to use

All necessary system ROM files used in machines and extensions are primarily identified with a checksum: a sha1sum. This enables openMSX to find the right ROM file from one of the file pools of type `system_rom`, regardless of the file name. So the actual content is guaranteed to be what was intended. If the ROM is explicitly specified in the configuration file (which is also supported) and the sha1sum doesn't match, a warning will be printed.

If you are trying to run an MSX machine and get an error like `Fatal error: Error in "broken" machine: Couldn't find ROM file for "MSX BIOS with BASIC ROM" (sha1: 12345c041975f31dc2ab1019cfdd4967999de53e).` it means that the required system ROM for that machine with the given sha1sum cannot be found in one of the file pools as mentioned above (typically `share/systemroms`). This is the primary way to know that you are missing required system ROMs and

therefore something went wrong installing them (typically either not a file with the proper content or you put the file in the wrong place, or you put it in a large ZIP file with multiple files).

The quickest way to see which machine and extensions work (i.e.: openMSX can find the required system ROMs the configuration is referring to) is by using the GUI under *Main menu bar → Machine → Test MSX hardware*. It will quickly check all machines and extensions and will show which are working and which are not, and which error occurred when trying to use it. Besides this, when selecting to run a machine using the GUI under *Main menu bar → Machine → Select MSX machine...*, you can also see which are working and which not and why. Likewise for extensions via *Main menu bar → Media → Extensions → Insert* for instance.

You can also manually check whether you have the correct ROM images. The value in the <sha1> tag(s) in the hardware configuration XML files contain checksums of ROM images that are known to work. You can compare the checksums of your ROM images to the ones in the hardware configuration XML files with the `sha1sum` tool. It is installed by default on most UNIX systems, on Windows you will have to download it separately. If the checksums match, it is almost certain you have correct system ROMs. If the checksums do not match, it could mean something went wrong dumping the ROMs, or it could mean you have a slightly older/newer model which contains different system ROMs.

A typical case in which you can have problems with checksums (or ROMs not getting found in a file pool) is disk ROMs. The ROM dump can be correct, and still have a different checksum. This is because part of the ROM is not actually ROM, but mapped on the registers of the floppy controller. When you are sure it is correct, don't put it in a file pool, but put it in the proper directory, which is explained above. Alternatively, you could add the checksum in the XML file that describes the machine you made the ROM dump for (multiple checksums can be present, they will be checked in the same order as they are in the file).

3.4.3 How to handle split ROMs

The machine configurations bundled with openMSX often refer to ROM files that span multiple 16 kB pages. For example, in the NMS 8250 configuration, the BIOS and MSX-BASIC are expected in a single 32 kB ROM image. If you created two 16 kB images when dumping or got those from downloading, you can concatenate them using tools included with your OS. In Linux and other Unix(-like) systems you can do it like this:

```
cat bios.rom basic.rom > nms8250_basic-bios2.rom
```

In Windows, open a command prompt and issue this command:

```
copy /b bios.rom + basic.rom nms8250_basic-bios2.rom
```

4. Palcom Laserdiscs

The Pioneer PX-7 and Pioneer PX-V60 are both emulated including an emulated Laserdisc Player, making it possible to run Palcom Laserdisc software.

The laserdisc must be captured before it can be used with an emulator. The file must adhere to the following rules:

- Use the Ogg container format
- Use the Vorbis codec for audio
- Use the Theora codec for video
- Captured at 640×480, YUV420
- A bitrate of at least 200kpbs for audio, otherwise the computer code encoded on the right audio channel will degrade too much for it to be readable
- Theora frame numbers must correspond to laserdisc frame numbers
- Some laserdiscs have chapters and/or stop frames. This is encoded in the VBI signal, and must be converted to plain text. This must be added to the Theora meta data

The metadata for chapters and stop frames has the form "chapter: <chapter-no>,<first-frame>-<last-frame>" and stop frames are "stop: <frame-no>". For example:

```
 chapter: 0,1-360 chapter: 1,361-4500 chapter: 2,4501-9450 chapter:
3,9451-18660 chapter: 4,18661-28950 chapter: 5,28951-38340 chapter:
6,38341-39432 stop: 4796 stop: 9089 stop: 9178 stop: 9751 stop: 14818 stop:
14908 stop: 18270 stop: 18360 stop: 18968 stop: 24815 stop: 24903 stop: 28553
stop: 28641 stop: 29258 stop: 34561 stop: 34649 stop: 38095 stop: 38181 stop:
38341 stop: 39127
```

Note that the emulated Pioneer PX-7 and Pioneer PX-V60 are virtually identical, except that the Pioneer PX-7 has pseudo-stereo for its PSG.

5. User Preferences

Almost all user preferences can be set via the GUI menu and the openMSX console, at openMSX run time. This is more thoroughly explained in the User's Manual.

By using the `bind` command you can create custom key bindings. These bindings will also be saved as settings in your settings file if you issue a `save_settings` command.

Many important settings are discussed in the User's Manual and there is an overview in the Console Command Reference.

If you're a power user and want to specify commands which are executed at the start of each openMSX start up, put those commands in a text file, one command per line (i.e. a script) and put it in the `share/scripts` directory. You can also explicitly specify a Tcl file to be loaded and executed on the openMSX command line. For this, use the `-script` command line option, which has the filename of the Tcl script as argument.

If you're a power user and want to tweak where openMSX reads and writes files from, you can use these hacky environment variables. Hacky, because we don't really expect anyone to change them, but when the urge is stronger than yourself, do so at your own risk... Be warned that they may change without notice in a next release.

variable	meaning
`OPENMSX_HOME`	The user's home folder, where all data will get stored that openMSX produces.
`OPENMSX_USER_DATA`	The user's personal `share` folder, where amongst others, system ROMs are searched
`OPENMSX_SYSTEM_DATA`	The system wide `share` folder in the openMSX installation directory

In the section about ROM locations you get an idea about the default values of these on different platforms.

6. Performance Tuning

This chapter contains some tips for tuning the performance of openMSX on your system.

6.1 OpenGL

As openMSX is using the SDLGL-PP renderer, it needs hardware acceleration to run at a decent speed, with support for OpenGL 2.0.

Getting OpenGL running hardware accelerated used to be a little cumbersome in some situations. However, nowadays there is a big chance that your system already has hardware accelerated OpenGL

supported in the default installation of your Xorg/Wayland or Windows environment.

You can verify hardware acceleration on your Linux system by typing `glxinfo` on the command line. If you have everything working, this command should output a line like this: `direct rendering: Yes.`

6.2 Various Tuning Tips

CPU and graphics performance varies a lot, depending on the openMSX settings and the MSX hardware and software you're emulating. Some things run fine on a 200MHz machine, others are slow on a 2GHz machine.

If openMSX is running slow, you can try the following measures:

- Disable the reverse feature (especially if the platform you're running on has a low amount of RAM), which is enabled by default on most platforms: `set auto_enable_reverse off`
- Make sure there are no CPU or I/O heavy background processes is running. Downloads, P2P software, distributed calculation efforts, search indexers etc. may grab the CPU from time to time, leaving less time for openMSX to do its job. Even if they only do so only once in a while, it may be enough to cause emulation to stutter.
- Increase the number of frames that may be skipped (`set maxframeskip 10`, for example).
- Use the blip resampler instead of the hq one.
- Emulate MSX software that uses fewer sound channels, for example MSX-MUSIC (maximum 9 channels) instead of MoonSound (maximum 18+24 channels). Or run simpler software altogether (e.g. MSX1 software instead of turboR software).

7. Writing Hardware Descriptions

There are two ways to use extra devices in your emulated MSX: you can use a shipped extension (which is similar to inserting a cartridge with the device into the MSX) or you can modify the hardware configuration file (the same as opening the MSX and building in the device). As in the real world, extensions are easier to use, but modifying the machine gives you more possibilities. Normal usage of machines and extensions is covered in the User's Manual; this chapter tells you how you can create or modify these hardware descriptions, which is a topic for advanced users and definitely something very few people will (want to) do. By editing the hardware configuration XML files, you can for example increase the amount of RAM, add built-in MSX-MUSIC, a disk drive, extra cartridge slots, etc.

You can modify an MSX machine (e.g. to add devices) by editing its hardware configuration XML file. So, let's make a copy of `share/machines/Philips_NMS_8250.xml` and put it in `share/machines/mymsx.xml`. It's the config we are going to play with; our custom MSX. Note: it is convenient to use the user directory (see above) to store your home-made machines, instead of the openMSX installation directory.

The easiest thing to do is to copy and modify fragments from other existing configurations that can be found in `share/machines` or `share/extensions`. For example, to add an FMPAC to the 8250, just copy it from the `share/extensions/fmpac.xml` to some place in your `mymsx.xml` file (between the `<devices>` and `</devices>` tags!):

```
<primary slot="2">
  <secondary slot="1">
    <FMPAC id="PanaSoft SW-M004 FMPAC">
      <io base="0x7C" num="2" type="O"/>
      <mem base="0x4000" size="0x4000"/>
      <sound>
        <volume>13000</volume>
        <balance>-75</balance>
      </sound>
      <rom>
        <sha1>9d789166e3caf28e4742fe933d962e99618c633d</sha1>
        <filename>roms/fmpac.rom</filename>
      </rom>
```

```
        <sramname>fmpac.pac</sramname>
      </FMPAC>
    </secondary>
  </primary>
```

Don't forget to add the `fmpac.rom` file to one of your `system_rom` file pools.

Because we changed the FMPAC from extension to built-in device, we have to specify in which slot the FMPAC is residing inside the modified 8250. So, we should replace the `slot="any"` stuff, with a specified slot as you can see in the above fragment. The number in the `slot` attribute of the `<primary>` tag indicates the primary slot of the emulated MSX you're editing. In this case the second cartridge slot of the NMS-8250 is used. `<secondary>` means sub slot. If we leave it out, the slot is not expanded and the primary slot is used. If we use it like in the above example, it means that slot 1 (of the `<primary>` tag) will be an expanded slot. If a `<primary>` tag has the attribute `external="true"`, this means that the slot is visible on the outside of the machine and can thus be used for external cartridges like extensions and ROM software. As explained above, the parameter filename can be adjusted to the name of your (64 kB!) FMPAC ROM file (note: if the file is not 65536 bytes in size, it won't work). "balance" defines to what channel the FMPAC's sound will be routed by default: in this case most of the sound goes to the left channel and a little bit goes to the right channel. "sramname" specifies the file name for file in which the SRAM contents will be saved to or loaded from. The saved files are compatible with the files that are saved by the (real) FMPAC commander's save option.

After saving your config and running openMSX again, you should be able to get the FMPAC commander with `CALL FMPAC` in the emulated MSX!

In a similar fashion, you can also add an MSX-Audio device (`<MSX-AUDIO>`, note that some programs also need the `MusicModuleMIDI` device to detect the Music Module, an empty SCC cartridge (`<SCC>`), etc. Just browse the existing extensions, check the Boosted_MSX2_EN configuration file and see what you can find.

Devices that contain ROM or RAM will have to be put inside a slot of the MSX, using the `<primary>` and `<secondary>` tags as demonstrated with the above mentioned FMPAC example. Other devices don't need this. Remember that you cannot put two devices that have a ROM in the same (sub)slot! Just use a new free subslot if you need to add such a device and all your primary slots are full. If a device does not need a slot, like the MSX-Audio device, you can add as many as you like.

Another thing you may want to change: the amount of RAM of the MSX: change the "size" parameter in the `<MemoryMapper>` device config.

In principle all of the above mentioned things are also valid for extensions. The main difference is the fact that you should use `"any"` for the slot specification as was already mentioned above. Just compare the fragment above with the original FMPAC extension we based it on.

If you understand the basics of XML, you should be able to compose your MSX now! You can use the ready-made configurations in `share/machines` as examples.

8. Contact Info

Because openMSX is still in heavy development, feedback and bug reports are very welcome!

If you encounter problems, you have several options:

1. Go to our IRC channel: `#openMSX on libera.chat` and ask your question there. Also reachable via webchat! If you don't get a reply immediately, please stick around for a while, or use one of the other contact options. The majority of the developers lives in time zone GMT+1. You may get no response if you contact them in the middle of the night...
2. Post a message on the openMSX forum on MRC.
3. Create a new issue in the openMSX issue tracker on GitHub. You need a (free) log-in on GitHub to get access.

4. Contact us and other users via one of the mailing lists. If you're a regular user and want to discuss openMSX and possible problems, join our `openmsx-user` mailing list. If you want to address the openMSX developers directly, post a message to the `openmsx-devel` mailing list. More info on the openMSX mailing lists, including an archive of old messages, can be found at SourceForge.

For experienced users: if you get a crash, try to provide a `gdb` backtrace. This will only work if you did not strip the openMSX binary of its debug symbols.

In any case, try to give as much information as possible when you describe your bug or request.